

GDB Cheatsheet (Linux)

Public visé : pentest / reverse · compatible GEF/Pwndbg · dernière maj : 2025-09

Démarrage & chargement

```
gdb ./prog                # binaire local
gdb --args ./prog -o out -v # avec arguments
gdb -q ./prog              # silencieux
sudo gdb -p <PID>          # attacher un processus
gdb ./prog core.*          # post-mortem (core dump)
set disassemble-next-line on # auto-désassemblage
```

Exécution & contrôle

```
run [args]                # lancer
start                     # lancer et stopper au main
continue / c              # reprendre
interrupt / Ctrl-C        # interrompre
kill                      # tuer l'inférieur
```

Pas-à-pas

```
next / n                  # step over (ligne)
step / s                  # step into (ligne)
ni                         # step over (instruction)
si                         # step into (instruction)
finish                    # sortir de la fonction
until <n>|*addr            # courir jusqu'à ligne/adresse
advance *addr              # comme until sans BP
```

Breakpoints

```
break main                # fonction
break file.c:42           # fichier:ligne
break *0x40123a            # adresse
rbreak regex              # BPs via regexp sur symboles

cond <id> expr             # condition
ignore <id> <count>       # ignorer N hits
```

```
tbreak <loc>                # temporaire (1 fois)
disable/enable <id>         # activer/désactiver
delete <id>|breakpoints     # supprimer

commands <id>               # actions auto au hit
> bt
> c
> end

info breakpoints            # lister
```

Breakpoints matériels (si support CPU)

```
hbreak *0x40123a           # HW breakpoint
```

Watchpoints (sur données)

```
watch var|*addr            # écriture
rwatch var|*addr           # lecture
awatch var|*addr           # lecture/écriture
info watchpoints
```

Fils d'exécution (threads)

```
info threads
thread <n>                  # changer de thread
thread apply all bt        # backtrace de tous
set schedule-multiple on   # laisser d'autres threads tourner entre BPs
```

Registres & flags

```
info registers
p/x $rax $rbx              # afficher
set $rip = 0x401000        # modifier
```

Mémoire & pile

```
x/<count><fmt><size> <addr> # examiner mémoire
# fmt: x(hex) d dec, i asm, s string, c char, f float, a addr
# size: b=1, h=2, w=4, g=8
x/16gx $rsp                # 16 QWORDS depuis la pile
x/s 0x7ffff7dd4000         # chaîne

bt / bt full                # backtrace
frame <n>                   # sélectionner la frame
info frame                  # détails frame
info args | info locals     # arguments / locals
```

Symboles & sources

```
info functions regex        # symboles fonctions
info variables regex        # symboles variables
list [file:line|func]       # afficher source
set substitute-path /build/src /usr/src # remapper chemins
```

Désassemblage & patch

```
disassemble <func>|*addr[,end]
set disassembly-flavor intel
x/8i $rip                  # 8 instructions
set {char}0x40123a = 0x90   # patch NOP
```

Signaux

```
info signals                # liste
handle SIGPIPE nostop noprint pass
handle SIGSEGV stop print nopass
```

Environnement & options runtime

```
set args -v -o out          # changer args
show args
set env F00=bar              # variables d'env
```

```
unset env F00
pwd / cd / shell <cmd>      # utilitaires
```

TUI (interface ncurses)

```
layout asm|src|split|regs
focus cmd|src|asm|regs
ctrl-x a      # bascule TUI/CLI
```

Core dumps (post-mortem)

```
ulimit -c unlimited
sysctl -w kernel.core_pattern='core.%e.%p.%t'
```

```
gdb ./prog core.*
bt full; info registers; x/32gx $rsp
disassemble $rip-32, $rip+32
```

Remote debugging

```
target remote :1234      # se connecter à gdbserver/qemu
set architecture i386:x86-64 # si nécessaire
monitor reset            # commandes cible (qemu/monitor)
```

Multi-binaires / inferiors

```
info inferiors
inferior <n>
add-inferior -exec ./autre
```

Expressions, formats, convenances

```
p expr      # print (décimal)
p/x expr    # hex
```

```
set print pretty on      # jolis tableaux/structs
set pagination off       # pas d'arrêt
```

Scripts & automatisation

```
source ~/gdbinit          # charger un script
set logging on            # journaliser la session
set logging file gdb.log

define hexd <addr> <len>
    x/<len>bx <addr>
end
```

ASLR / PIE — repères

```
cat /proc/sys/kernel/randomize_va_space # 2 par défaut
setarch $(uname -m) -R ./prog           # exécuter sans ASLR
readelf -hW a.out | grep Type:          # DYN => PIE
```

Extensions utiles

- **Pwndbg**: vmmap, telescope, rop, canary, visheap
- **GEF**: gef config, distance, heap bins, pattern create 200
- **PEDA**: pdisas, dumpargs, telescope

Scénarios rapides (recettes)

Trouver qui écrase un buffer

```
break vuln
run AAAA
awatch buf
continue # stop à la 1re écriture
bt
```

Ignorer les 100 premiers hits d'un BP bruyant

```
break *0x401000
ignore 1 100
continue
```

Déboguer un crash post-mortem

```
gdb ./srv core.srv.*
bt full
info registers
x/64gx $rsp
```

Suivre un thread spécifique

```
info threads
thread 5
thread apply 5 bt
```

Raccourcis à retenir

- `n/ni` = step over ligne/inst · `s/si` = step into
- `bt` = pile · `i r` = registres · `x/...` = mémoire
- `b ...` = breakpoint · `c` = continue · `fin` = finish

Astuce QS

Active `set disassemble-next-line on` et installe **Pwndbg** : le flux ASM/source + états de registres accélère l'analyse en CTF/RE.